Enterprise Selenium Java BDD Automation Framework

This document describes an enterprise-grade Selenium Automation Framework using: Java Selenium WebDriver BDD with Cucumber TestNG Maven Extent Reports Hybrid Execution Support (Local/Grid/Cloud) Jenkins CI/CD

1. Framework Architecture

```
selenium-java-bdd-framework/
│
├── pom.xml
├── testng.xml
├── Jenkinsfile
├── README.md
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── framework/
│   │   │       ├── base/
│   │   │       ├── config/
│   │   │       ├── constants/
│   │   │       ├── drivers/
│   │   │       ├── pages/
│   │   │       ├── utilities/
│   │   │       ├── reports/
│   │   │       ├── listeners/
│   │   │       ├── api/
│   │   │       └── database/
│   │   │
│   │   └── resources/
│   │       ├── log4j2.xml
│   │       ├── extent-config.xml
│   │       └── testdata/
│   │
│   └── test/
│       ├── java/
│       │   ├── runners/
│       │   ├── stepdefinitions/
│       │   └── tests/
│       │
│       └── resources/
│           └── features/
│
├── reports/
├── screenshots/
├── logs/
└── drivers/
```

2. Technology Stack

Layer	Technology
Programming Language	Java
Automation Tool	Selenium WebDriver
BDD Framework	Cucumber
Test Framework	TestNG
Build Tool	Maven
Reporting	Extent Reports

Logging	Log4j2
CI/CD	Jenkins
Execution	Local + Grid + Cloud

Step 1 — Project Initialization

- Maven project creation
- Selenium dependency setup
- Cucumber integration
- TestNG integration
- Initial runner configuration
- Maven Surefire plugin setup

Step 2 — Configuration Management

- Environment property files
- ConfigReader implementation
- Multi-environment execution
- Browser configuration handling

Step 3 — Driver Management

- DriverFactory implementation
- ThreadLocal WebDriver
- Local execution support
- Selenium Grid support
- BrowserStack/SauceLabs support

Step 4 — Page Object Model

- BasePage creation
- Reusable Selenium wrappers
- Page classes implementation
- Element synchronization

Step 5 — BDD Layer

- Feature files
- Step Definitions
- Hooks implementation
- Scenario tagging

Step 6 — Reporting & Logging

- Extent Reports integration
- Screenshot capture
- Failure analysis
- Log4j2 implementation

Step 7 — Utilities Layer

- WaitUtils
- RetryAnalyzer
- ExcelUtils
- JavaScriptUtils
- FileUtils

Step 8 — CI/CD Integration

```
- Jenkins pipeline
- Parallel execution
- Scheduled execution
- Report archiving
```

Step 9 — Advanced Enhancements

```
- Docker integration
- Selenium Grid scaling
- API automation support
- Database validation
- Cross-browser execution
```

3. Sample pom.xml Dependencies

```
<dependencies>

    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.21.0</version>
    </dependency>

    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-java</artifactId>
        <version>7.18.0</version>
    </dependency>

    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-testng</artifactId>
        <version>7.18.0</version>
    </dependency>

    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.10.2</version>
    </dependency>

    <dependency>
        <groupId>io.github.bonigarcia</groupId>
        <artifactId>webdrivermanager</artifactId>
        <version>5.8.0</version>
    </dependency>

    <dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>5.1.1</version>
    </dependency>

</dependencies>
```

4. Enterprise Best Practices

Use ThreadLocal for parallel execution. Never hardcode locators or test data. Maintain reusable utilities. Use explicit waits over implicit waits. Capture screenshots for failures. Separate environment configurations. Maintain proper logging standards. Use tags for regression/smoke/sanity execution. Integrate framework with CI/CD pipeline. Implement retry logic for flaky tests.

This framework is designed for scalability, maintainability, enterprise-level reporting, parallel execution, and cloud-ready automation execution.